

PROPOSED PROJECT STRUCTURE

AI-Native Board Defect Inspection System

Same architecture as the beginner guide • Inspection domain • Eleven layers • Six quality gates • Edge and enterprise

CONTENTS

- | | |
|---|--|
| 1. What stays the same, what the domain changes | 7. Quality gates and where they are enforced |
| 2. The four architecture views, applied to inspection | 8. The three API surfaces |
| 3. Proposed project structure | 9. Structural rules this layout protects |
| 4. Directory reference | 10. What is deliberately not in the tree |
| 5. Runtime to directory map | 11. Open decisions before committing |
| 6. Eleven architecture layers to directory map | 12. How to validate the structure |

1. What stays the same, what the domain changes

The architecture from the beginner guide carries over unchanged. Frontend and backend remain the system and deployment view; presentation, domain, AI and data remain the logical view; UI Runtime, Domain Runtime and AI Runtime remain the AI-native runtime view; and harness plus Claude Code plus tests plus evals plus feedback remain the engineering layer. The rule that AI recommends while the Domain Runtime decides is unchanged, and the Generative UI principle that a model selects from an approved component vocabulary rather than emitting executable code is unchanged.

Three things the domain does *not* change: the runtime boundaries, the deterministic-versus-probabilistic split, and the position of Claude Code in engineering time rather than in the product runtime.

What the domain does change is deployment topology and the shape of the users. Four consequences follow, and they are the only reasons this tree differs from the one in the beginner guide.

- The same three runtimes are deployed twice. An inspection system has a factory edge tier that must keep working without the enterprise, so UI, Domain and AI Runtime code is composed into two deployment targets rather than one. This is a deployment concern, not a second architecture, which is why `edge/` and `enterprise/` hold composition only and never business logic.
- There is no customer. The UI Runtime serves operators, inspectors and quality managers, so Generative UI shifts from landing-page composition to role-aware review and inspection surfaces, and the approved component catalogue changes accordingly.
- The Domain Runtime gains line equipment as a consumer. PLC, MES and quality-system adapters exist, and no inference may reach line equipment without passing the decision path.
- The AI Runtime is vision-first rather than language-first, which makes the model lifecycle a substantial engineering area in its own right. That is the one genuinely new top-level area: `m1/`.

2. The four architecture views, applied to inspection

VIEW	MAIN QUESTION	APPLIED TO INSPECTION
System / deployment	What is deployed and talks to what?	Operator web frontend, backend services, AI services, data stores, line adapters — deployed as an edge composition and an enterprise composition
Logical	What responsibilities exist?	Presentation, application and workflow, domain and business rules, AI capabilities, data and infrastructure
AI-native runtime	What executes together at runtime?	UI Runtime, Domain Runtime, AI Runtime, data and services — the same three, present in both tiers
Technology	What implements it?	Illustrative only: web frontend in React and TypeScript, backend and AI services in Python. Not yet decided — see section 11.
Engineering	What helps us build it safely?	Harness, Claude Code, tests, evals, feedback

3. Proposed project structure

```

board-defect-inspection/
|
|-- CLAUDE.md                how Claude Code works in this project
|-- README.md
|
|-- .claude/                 HARNESS (engineering time)
|   |-- rules/               project rules and constraints
|   |-- hooks/               enforced actions, pre/post checks
|   |-- agents/              specialized agent instructions
|   `-- settings.json        permissions, allowed tools
|
|-- docs/                    HARNESS: shared understanding
|   |-- business-problem.md   derived from the BRD
|   |-- architecture.md       the four views, edge and enterprise
|   |-- quality-gates.md      QG-1 ... QG-6 and failure actions
|   |-- security.md
|   `-- adr/                  numbered decision records
|
|-- specs/                   HARNESS: what to build
|   |-- spec.md               functional, NFR, AI requirements
|   |-- model-contracts.md     per-agent contracts and limits
|   |-- backend-services.md    service boundaries
|   |-- api-contracts.md       line, review and admin surfaces
|   |-- data-model.md          evidence, records, cache, registry
|   `-- ui-contract.md         Generative UI specification schema
|
|-- policies/                BUSINESS-OWNED, versioned separately
|   |-- defect-taxonomy.md     D001 ... D006, severities
|   |-- thresholds.md          per-defect recall, review bands
|   |-- decision-rules.md      BR-001 ... BR-005
|   `-- ui-layout-policy.md    what Generative UI may compose
|
|-- plans/implementation-plan.md
|-- tasks/tasks.md
|
|-- contracts/                SHARED SCHEMAS (cross-runtime, cross-tier)
|   |-- inspection/           attempt, finding, decision record
|   |-- ui/                   UI specification schema
|   `-- events/               sync and outbox message shapes
|
|-- frontend/                 UI RUNTIME
|   |-- src/
|   |   |-- views/            operator, inspector, manager
|   |   |-- components/approved/ approved component catalogue
|   |   |-- renderer/          generative UI renderer + validation

```

```

| | |-- fallback/      standard UI when generation fails
| | |-- `-- api/       typed clients from contracts/
| | |-- `-- tests/
|
|-- backend/           DOMAIN RUNTIME
| |-- api/
| | |-- |-- line/      machine to machine, tight latency
| | |-- |-- review/    human facing, attributed actions
| | |-- |-- `-- admin/  privileged, separately audited
| | |-- |-- workflow/   inspection state machine
| | |-- |-- gates/      QG-1 ... QG-6 enforcement
| | |-- |-- rules/      policy evaluation, BR-001 ... BR-005
| | |-- |-- services/   one module per architecture layer
| | |-- |-- adapters/   PLC, MES, quality, capture source
| | |-- |-- persistence/ evidence, records, cache, registry
| | |-- |-- security/   authn, authz, override authority
| | |-- |-- audit/      immutable trail
| | |-- |-- `-- tests/
|
|-- ai/                AI RUNTIME
| |-- gateway/         model invocation, version pinning
| |-- vision/          understand, detect, classify, localize
| |-- calibration/     scores -> probabilities
| |-- risk/            severity, criticality, uncertainty
| |-- uigen/           UI specification generation
| |-- prompts/         versioned prompt assets
| |-- context/         board spec and BOM context assembly
| |-- `-- tests/
|
|-- ml/                MODEL LIFECYCLE (architecture layer 8)
| |-- preprocessing/   SHARED with ai/vision at serving time
| |-- training/
| |-- experiments/
| |-- packaging/       model package, target hardware build
| |-- datasets/        manifests and pointers, never data
| |-- `-- registry/    approval and release records
|
|-- edge/              EDGE DEPLOYMENT COMPOSITION
| |-- compose/         which services run on the line
| |-- config/          line, camera, inspection point
| |-- `-- offline/     cache validity, outbox behaviour
|
|-- enterprise/        ENTERPRISE DEPLOYMENT COMPOSITION
| |-- compose/
| |-- `-- config/
|
|-- simulators/        HARNESS: exercise without a factory
| |-- capture/         synthetic and recorded image source

```

```
| |-- plc/          signal, acknowledgement, state checks
| |-- mes/
| `-- offline-mode/ enterprise unreachable
|
|-- evals/          HARNESS: probabilistic verification
| |-- datasets/     golden cases, synthetic and held out
| |-- expected/
| |-- scenarios/     poor image, unknown variant, latency
| |-- runner/        executable, wired into CI
| `-- evaluation-rubric.md
|
|-- tests/          cross-cutting and end to end
|-- ops/            runbooks, dashboards, alerts, SLOs
|-- infra/          environments, deployment, secrets policy
`-- ci/             gate checks, eval runs, release gating
```

4. Directory reference

DIRECTORY	HOLDS	TIER
<code>.claude/</code>	Harness enforcement: rules, hooks, agent instructions, permissions. The part of the harness that acts rather than advises.	Engineering
<code>docs/</code>	Business problem, architecture, quality gates, security, and numbered decision records.	Engineering
<code>specs/</code>	What to build, including model contracts, service boundaries, API surfaces, data model and the UI specification schema.	Engineering
<code>policies/</code>	Quality-owned taxonomy, thresholds, decision rules and UI layout policy. Versioned independently of code.	Business
<code>contracts/</code>	Schemas shared across runtimes and across tiers. Sits above frontend, backend and ai so none of them owns the wire format.	Both
<code>frontend/</code>	UI Runtime: role-aware views, the approved component catalogue, the generative renderer with its validation, and the standard fallback UI.	Both
<code>backend/</code>	Domain Runtime: three API surfaces, workflow state, gate enforcement, policy evaluation, adapters, persistence, security and audit.	Both
<code>ai/</code>	AI Runtime: model gateway, vision pipeline, calibration, risk, UI generation, prompts and context assembly.	Both
<code>ml/</code>	Model lifecycle: shared preprocessing, training, experiments, packaging for target hardware, dataset manifests, approval records.	Enterprise
<code>edge/</code>	Composition and configuration only: which services run on the line, cache validity, outbox behaviour.	Edge
<code>enterprise/</code>	Composition and configuration for the central platform.	Enterprise
<code>simulators/</code>	Capture source, PLC, MES and offline mode, so the pipeline and the coding agent can run without a factory.	Engineering
<code>evals/</code>	Golden datasets, expected behaviour, scenarios, the executable runner and the rubric.	Engineering

DIRECTORY	HOLDS	TIER
ops/ , infra/ , ci/	Runbooks and alerts; environments and deployment; gate checks, eval runs and release gating.	Engineering

5. Runtime to directory map

RUNTIME	PRIMARY DIRECTORY	RESPONSIBILITIES IN THIS DOMAIN
UI Runtime	frontend/	Deterministic: authentication, navigation, override controls, approval actions. Generative: role-aware review panels, pinned active reviews, contextual explanation of a decision.
Domain Runtime	backend/	Inspection workflow and state, quality gates, decision policy, board identity, PLC and MES signalling, override authority, evidence persistence, audit, retention, release gating.
AI Runtime	ai/	Defect detection, classification and localization, anomaly and out-of-distribution scoring, calibration, uncertainty, severity assessment, UI specification generation.
Data / services	backend/persistence/ , infra/	Evidence store, append-only decision records, context store and edge cache, label store, model registry, immutable audit log.
Harness	.claude/ , docs/ , specs/ , evals/ , simulators/	Context, constraints, tools, verification and feedback for Claude Code.

6. Eleven architecture layers to directory map

#	LAYER	LIVES IN	GATE
1	Data capture and ingestion	backend/services/ , backend/adapters/capture	QG-1
2	Edge AI processing	ai/vision/ , ml/preprocessing/ , edge/compose/	QG-2
3	Context and reference	ai/context/ , backend/persistence/	QG-3
4	AI model layer	ai/gateway/ , ai/vision/ , ml/packaging/	—
5	Probabilistic reasoning and risk	ai/calibration/ , ai/risk/	—
6	Decision layer	backend/rules/ , backend/workflow/	QG-4
7	Human in the loop	backend/api/review/ , frontend/views/	QG-5
8	Model lifecycle and MLOps	ml/ , evals/ , ci/	QG-6
9	Observability and operations	ops/ , infra/	—
10	Governance, security, compliance	backend/security/ , backend/audit/ , policies/	—
11	Adaptive UI	frontend/renderer/ , ai/uigen/ , policies/ui-layout-policy.md	—

Layer 11 is where the beginner guide's Generative UI principle lands. The model produces a UI specification in `ai/uigen/` against the schema in `contracts/ui/`; `frontend/renderer/` validates it and maps approved types to real components; `frontend/fallback/` takes over on invalid output or timeout. The model composes presentation, the user authorizes the action, and `backend/security/` enforces authorization regardless of what was rendered.

7. Quality gates and where they are enforced

GATE	QUESTION	ENFORCED IN	FAILURE ACTION
QG-1	Is the input valid and usable?	backend/gates/	Quarantine or bounded re-capture
QG-2	Is inference healthy and in limits?	backend/gates/	HOLD or bounded re-process
QG-3	Is board and line context available?	backend/gates/	HOLD or exception
QG-4	Is the decision justified?	backend/rules/	HOLD for human review
QG-5	Are review labels reliable?	backend/api/review/	Escalate
QG-6	Is the model evaluated and approved?	ci/ with ml/registry/	Iterate; no self-approved release

Gates live in the Domain Runtime, never in the AI Runtime. A gate implemented inside ai/ would be a model checking its own work.

8. The three API surfaces

SURFACE	DIRECTORY	CONSUMERS	CHARACTER
Line facing	backend/api/line/	Capture source, PLC, MES	Machine to machine, tight latency budget, highest availability, no interactive authentication
Human facing	backend/api/review/	Operators, inspectors, managers	Interactive authentication, role-aware authorization, every mutation attributed to a named user
Administrative	backend/api/admin/	Quality, AI and platform owners	Most privileged: taxonomy, thresholds, rules, model approval. Separately authenticated and audited, unreachable from the line surface

9. Structural rules this layout protects

Each of these is a reason a directory exists where it does. If any of them is violated, the structure has stopped helping.

- `ml/preprocessing/` is imported by `ai/vision/` at serving time. Preprocessing must exist once, not once in training and again in inference — duplicated preprocessing is the classic cause of training and serving skew, and it surfaces as unexplained recall loss in production rather than as a failing test.
- `contracts/` sits above the three runtimes so that an edge decision and its later central re-analysis remain comparable, and so no runtime owns the wire format.
- `policies/` is separate from code because thresholds and taxonomy are Quality-owned. The repository copy is the seed and default; the runtime source of truth is the policy service behind the admin surface, so a threshold change is an audited business action rather than a deployment.
- `backend/adapters/` is the only place permitted to talk to line equipment, which keeps protocol detail out of the decision path and makes every integration testable against `simulators/`.
- `simulators/` exists because a coding agent with no way to exercise degraded capture, a latency breach or a disconnected enterprise will write happy-path code with confidence.
- `evals/runner/` is executable and wired into `ci/`. A rubric that is not run decays within weeks.
- `edge/` and `enterprise/` hold composition only. Inspection logic accumulating there means the service boundaries were drawn in the wrong place.

No directory layout enforces anything. Three invariants in particular need tests that prove they cannot be bypassed, not folders that suggest it:

- No path from inference to line equipment that skips the decision path
- Evidence persisted before any action is taken on a board
- A HOLD is never resolved by timeout — an expiring HOLD turns a capacity problem into a quality escape

10. What is deliberately not in the tree

- Image and training data. `ml/datasets/` holds manifests and pointers; the data itself lives in versioned external storage with retention policy applied.

- Model binaries. `ml/registry/` holds approval and release records; packages live in the registry the deployment reads from.
- Secrets. `infra/` holds the policy for handling them, never values.
- Labelling tooling. Referenced through the review surface and the label store rather than vendored into the repository.
- Dependency manifests, build files and pipeline definitions, because the technology decision is still open. Those appear the moment section 11 is settled.

11. Open decisions before committing

In priority order, because the first two change the shape of the tree and the rest only change its contents.

1. **Deployment granularity.** `backend/services/` is a logical decomposition, not nine deployable services. For a first build, one deployable with enforced internal module boundaries is almost certainly right, extracting later where throughput or release cadence demands it. Decide this explicitly before anyone reads the tree as a microservice mandate.
2. **Repository strategy.** Edge devices and the enterprise platform will not ship on the same cadence. A single repository works initially, but release tagging per component is needed early, and a later split of the edge tier should be anticipated rather than discovered.
3. **Technology stack.** The beginner guide names React, TypeScript, Python, FastAPI and PostgreSQL. Those are illustrative here. The BRD defers infrastructure and deployment, and the vision workload plus the edge hardware target should drive the choice. Record it in `docs/adr/`.
4. **Policy runtime source of truth.** Confirm that `policies/` is the seed and the policy service is authoritative at runtime, and define how a policy version is pinned into a decision record.
5. **Edge decision autonomy.** Whether layers 5 and 6 are approved to run at the edge for offline continuity determines how much of `ai/` and `backend/` ships to the line, and it depends on the unresolved answer to what happens when the enterprise is unreachable.

12. How to validate the structure

Do not accept this layout on paper. Build the first vertical slice through it — image capture, QG-1 enforcement, evidence persistence via the capture source adapter and simulator, exposed only on the

line surface — and watch for files that want a home the tree does not offer. That single exercise settles the deployment granularity question and the policy source-of-truth question faster than further design discussion, and it is cheap to redraw a directory at that point.

Two further notes on provenance. This structure is a synthesis of the beginner guide's architecture, the BRD and the proposed eleven-layer design; it is not a validated reference architecture, and the design it rests on is itself marked as requiring validation. And the recommended first model-driven feature differs from the beginner guide by necessity: with no customer-facing landing page in this domain, the equivalent small, visible Generative UI demonstration is the role-aware inspector review panel, which exercises the same renderer, approved catalogue and fallback path.

Proposed project structure for the AI-Native Board Defect Inspection System. Review draft — architecture inherited from 02-AI_Native_Engineering_Architecture_Beginner_Guide; domain and layers from the BRD and the Enterprise AI-Native PCB Defect Inspection Architecture.